

Core Language Working Group "ready" Issues

Section references in this document reflect the section numbering of document [WG21 N4527](#).

1274. Common nonterminal for *expression* and *braced-init-list*

Section: 6.5.4 [stmt.ranged] **Status:** ready **Submitter:** Daniel Krügler **Date:** 2011-03-25

It would be helpful to have a single grammar term for *expression* and *braced-init-list*, which often occur together in the text. In particular, 6.5.4 [stmt.ranged] paragraph 1 allows both, but the description of `__RangeT` refers only to the *expression* case; such errors would be less likely if the common term were available.

Proposed resolution (May, 2015):

1. Add a new production to the grammar in 8.5 [dcl.init] paragraph 1:

```
expr-or-braced-init-list:
    expression
    braced-init-list
```

2. Change the grammar in 5.2 [expr.post] paragraph 1 as follows:

```
postfix-expression:
    primary-expression
    postfix-expression [ expression expr-or-braced-init-list ]
    postfix-expression { braced-init-list }
    postfix-expression ( expression-listopt )
    ...
```

3. Change the grammar in 6.5 [stmt.iter] paragraph 1 as follows:

```
for-range-initializer:
    expression
    braced-init-list
    expr-or-braced-init-list
```

4. Change 6.5.4 [stmt.ranged] paragraph 1 as follows:

~~For a range-based `for` statement of the form~~

~~`for` (*for-range-declaration* : *expression*) *statement*~~

~~let *range-init* be equivalent to the *expression* surrounded by parentheses⁹⁰~~

~~(*expression*)~~

~~and for a range-based `for` statement of the form~~

~~`for` (*for-range-declaration* : *braced-init-list*) *statement*~~

~~let *range-init* be equivalent to the *braced-init-list*. In each case, a~~ **A** ~~range-based `for` statement is equivalent to~~

```
{
    auto && __range = range-init for-range-initializer;
    for ( auto __begin = begin-expr,
          __end = end-expr;
          __begin != __end;
          ++__begin ) {
        for-range-declaration = *__begin;
        statement
    }
}
```

where

- if the *for-range-initializer* is an *expression*, it is regarded as if it were surrounded by parentheses (so that a comma operator cannot be reinterpreted as delimiting two *init-declarators*);
- `__range`, `__begin`, and `__end` are variables defined for exposition only;
- `__RangeT` is the type of the expression, and
- *begin-expr* and *end-expr* are determined as follows:
 - if `__RangeT` the *for-range-initializer* is an *expression of* array type `R`, *begin-expr* and *end-expr* are `__range` and `__range + __bound`, respectively, where `__bound` is the array bound. If `__RangeT R` is an array of unknown size or an array of incomplete type, the program is ill-formed;
 - if `__RangeT` the *for-range-initializer* is a *an expression of* class type `C`, the *unqualified-ids* *begin* and *end* are looked up in the scope of `class __RangeT C` as if by class member access lookup (3.4.5 [basic.lookup.classref]), and if either (or both) finds at least one declaration, *begin-expr* and *end-expr* are `__range.begin()` and `__range.end()`, respectively;
 - otherwise, *begin-expr* and *end-expr* are `begin(__range)` and `end(__range)`, respectively, where *begin* and *end* are looked up in the associated namespaces (3.4.2 [basic.lookup.argdep]). [Note: Ordinary unqualified lookup (3.4.1 [basic.lookup.unqual]) is not performed. —end note]

5. Change the grammar of 6.6 [stmt.jump] paragraph 1 as follows:

```
jump-statement:
    break ;
    continue ;
    return expression expr-or-braced-init-listopt ;
return braced-init-list ;
    goto identifier ;
```

6. Change 6.6.3 [stmt.return] paragraph 2 as follows:

The *expression-or-braced-init-list* *expr-or-braced-init-list* of a return statement is called its operand. A return statement...

7. Change 13.5.5 [over.sub] paragraph 1 as follows:

`operator[]` shall be a non-static member function with exactly one parameter. It implements the subscripting syntax

```
postfix-expression [ expression expr-or-braced-init-list ]
```

or

```
postfix-expression { braced-init-list }
```

Thus, a subscripting expression...

1391. Conversions to parameter types with non-deduced template arguments

Section: 14.8.1 [temp.arg.explicit] **Status:** ready **Submitter:** Jason Merrill **Date:** 2011-09-08

According to 14.8.1 [temp.arg.explicit] paragraph 6,

Implicit conversions (Clause 4 [conv]) will be performed on a function argument to convert it to the type of the corresponding function parameter if the parameter type contains no *template-parameters* that participate in template argument deduction. [Note: Template parameters do not participate in template argument deduction if they are explicitly specified...

But this isn't clear about when these conversions are done. Consider

```
template<class T> struct A {
    typename T::N n;
};
template<class T> struct B { };

template<class T, class T2>
void foo(const A<T>& r); // #1
template<class T>
void foo(const B<T>& r); // #2

void baz() {
    B<char> b;
    foo(b); // OK
    foo<char>(b); // error
}
```

With the explicit template argument, the first parameter of #1 no longer participates in template argument deduction, so implicit conversions are done. If we check for the implicit conversion during the deduction process, we end up instantiating `A<char>`, resulting in a hard error. If we

wait until later to check the conversion, we can reject #1 because τ_2 is not deduced and never need to consider the conversion.

But if we just accept the parameter and leave it up to normal overload resolution to reject an unsuitable candidate, that breaks this testcase:

```
template<class T>
struct A {
    typename T::N n;
};

template<class T>
struct B { };

template <class T, class... U>
typename A<T>::value_t bar(int, T, U...);

template <class T>
T bar(T, T);

void baz()
{
    B<char> b;
    bar(b, b);
}
```

Here, if deduction succeeds, we substitute in the deduced arguments of $\tau = B<char>$, $U = \{ \}$, and end up instantiating $A<B<char>>$, which fails.

EDG and GCC currently reject the first testcase and accept the second; clang accepts both.

Notes from the October, 2012 meeting:

The position initially favored by CWG was that implicit conversions are not considered during deduction but are only applied afterwards, so the second example is ill-formed, and that the normative wording of the referenced paragraph should be moved into the note. This approach does not handle some examples currently accepted by some implementations, however; for example:

```
template <class T> struct Z {
    typedef T::x xx;
};
template <class T> Z<T>::xx f(void *, T);
template <class T> void f(int, T);
struct A {} a;
int main() {
    f(1, a); // If the implementation rules out the first overload
            // because of the invalid conversion from int to void*,
            // the error instantiating Z<A> will be avoided
}
```

Additional discussion is required.

Notes from the April, 2013 meeting:

The approach needed to accept this code appears to be doing the convertibility check between deduction and substitution.

Proposed resolution (May, 2015):

1. Change 14.8.2.1 [temp.deduct.call] paragraph 1 as follows:

Template argument deduction is done by comparing each function template parameter type (call it P) **that contains template-parameters that participate in template argument deduction** with the type of the corresponding argument of the call (call it A) as described below. If P is a dependent type, removing references and cv-qualifiers from P gives $\text{std::initializer_list}<P'\>$ or $P'[N]$ for some P' and N and the argument is a non-empty initializer list (8.5.4 [dcl.init.list]), then deduction is performed instead for each element of the initializer list, taking P' as a function template parameter type and the initializer element as its argument, and in the $P'[N]$ case, if N is a non-type template parameter, N is deduced from the length of the initializer list. Otherwise, an initializer list argument causes the parameter to be considered a non-deduced context (14.8.2.5 [temp.deduct.type]). [Example:...

2. Delete the note in 14.8.2.1 [temp.deduct.call] paragraph 4:

~~[Note: as specified in 14.8.1 [temp.arg.explicit], implicit conversions will be performed on a function argument to convert it to the type of the corresponding function parameter if the parameter contains no template-parameters that participate in template argument deduction. Such conversions are also allowed, in addition to the ones described in the preceding list. —end note]~~

3. Add the following as a new paragraph at the end of 14.8.2.1 [temp.deduct.call]:

If deduction succeeds for all parameters that contain template-parameters that participate in template argument deduction, and all template arguments are explicitly specified, deduced, or obtained from default template arguments, remaining parameters are then compared with the corresponding arguments. For each remaining parameter P with a type that was non-dependent before substitution of any explicitly-specified template arguments, if the corresponding argument A cannot be implicitly converted to P , deduction fails. [Note: Parameters with dependent types in which no template-parameters participate in template argument deduction, and parameters that became non-dependent due to substitution of explicitly-specified template arguments, will be checked during overload resolution. —end note]

[Example:

```
template <class T> struct Z {
    typedef typename T::x xx;
};
template <class T> typename Z<T>::xx f(void *, T); // #1
template <class T> void f(int, T); // #2
struct A {} a;
int main() {
    f(1, a); // OK, deduction fails for #1 because there is no conversion from int to void*
}
```

—end example]

4. Change 14.8.2.4 [temp.deduct.partial] paragraph 4 as follows:

Each type nominated above from the parameter template and the corresponding type from the argument template are used as the types of *P* and *A*. **If a particular *P* contains no *template-parameters* that participate in template argument deduction, that *P* is not used to determine the ordering.**

5. Change 14.8.2.5 [temp.deduct.type] paragraph 4 as follows:

In most cases, the types, templates, and non-type values that are used to compose *P* participate in template argument deduction. That is, they may be used to determine the value of a template argument, and the value so determined must be consistent with the values determined elsewhere. In certain contexts, however, the value does not participate in type deduction, but instead uses the values of template arguments that were either deduced elsewhere or explicitly specified. If a template parameter is used only in non-deduced contexts and is not explicitly specified, template argument deduction fails. **[Note: Under 14.8.2.1 [temp.deduct.call] and 14.8.2.4 [temp.deduct.partial], if *P* contains no *template-parameters* that appear in deduced contexts, no deduction is done, and so *P* and *A* need not have the same form. —end note]**

This resolution also resolves issue 1847.

Additional note October, 2015:

See also issue 1939.

1722. Should lambda to function pointer conversion function be `noexcept`?

Section: 5.1.2 [expr.prim.lambda] **Status:** ready **Submitter:** Ville Voutilainen **Date:** 2013-07-31

According to 5.1.2 [expr.prim.lambda] paragraph 6,

The closure type for a non-generic *lambda-expression* with no *lambda-capture* has a public non-virtual non-explicit `const` conversion function to pointer to function with C++ language linkage (7.5 [dcl.link]) having the same parameter and return types as the closure type's function call operator.

This does not specify whether the conversion function is `noexcept(true)` or `noexcept(false)`. It might be helpful to nail that down.

Proposed resolution (May, 2015):

Change 5.1.2 [expr.prim.lambda] paragraph 6 as follows:

The closure type for a non-generic *lambda-expression* with no *lambda-capture* has a **public non-virtual non-explicit `const`** conversion function to pointer to function with C++ language linkage (7.5 [dcl.link]) having the same parameter and return types as the closure type's function call operator. The value returned by this conversion function shall be the address of a function that, when invoked, has the same effect as invoking the closure type's function call operator. For a generic lambda with no *lambda-capture*, the closure type has a **public non-virtual non-explicit `const`** conversion function template to pointer to function. The conversion function template... [Example:

```
auto GL = [](auto a) { std::cout << a; return a; };
int (*GL_int)(int) = GL; // OK: through conversion function template
GL_int(3); // OK: same as GL(3)
```

—end example] **The conversion function or conversion function template is public, non-virtual, non-explicit, `const`, and has a non-throwing exception specification (15.4 [except.spec]).**

1847. Clarifying compatibility during partial ordering

Section: 14.8.2.5 [temp.deduct.type] **Status:** ready **Submitter:** Jason Merrill **Date:** 2014-01-30

There is implementation divergence in the handling of an example like

```
template<typename D> struct A { };
template<typename T> struct Wrap1 { typedef T type; };
template<typename T> struct Wrap2 { typedef T type; };

template<typename T1>
A<typename Wrap1<T1>::type>
fn(const A<T1>& x, const A<T1>& y);

template<typename T2, typename U>
A<typename Wrap2<T2>::type>
fn(const A<T2>& x, const A<U>& y);

A<int> (*p)(const A<int>&, const A<int>&) = fn;
```

The implementations that accept this example do so by not comparing the return types of the two templates during partial ordering, which seems to make sense given that partial ordering would not have been performed if the candidate specializations were not indistinguishable from the perspective of overload resolution. However, the existing wording is not clear that that is how such types are handled.

Proposed resolution (May, 2015):

This issue is resolved by the resolution of issue 1391.

1863. Requirements on thrown object type to support `std::current_exception()`

Section: 15.1 [except.throw] **Status:** ready **Submitter:** Canada **Date:** 2013-09-22 [N3690 comment CA 12](#)

The specification of `std::current_exception()` in 18.8.5 [propagation] allows either referring to the exception object itself or to a copy thereof, implying that the exception object must be copyable. However, the specification of *throw-expression* allows throwing objects that cannot be copied, only moved. Presumably the requirements should include a non-deleted accessible copy constructor that is odr-used by a *throw-expression*, even if the object being thrown is moved to the exception object.

Additional note, February, 2014:

This issue was referred to CWG by EWG at the September, 2013 meeting but was overlooked at that time.

Proposed resolution (May, 2015):

1. Change 12.4 [class.dtor] paragraph 11 as follows:

...A destructor is potentially invoked if it is invoked or as specified in 5.3.4 [expr.new] **and**, 12.6.2 [class.base.init], **and 15.1 [except.throw]**. A program is ill-formed if...

2. Change 15.1 [except.throw] paragraph 5 as follows:

When the thrown object is a class object, the constructor selected for the copy-initialization **as well as the constructor selected for a copy-initialization considering the thrown object as an lvalue** ~~and the destructor~~ shall be **non-deleted and** accessible, even if the copy/move operation is elided (12.8 [class.copy]). **The destructor is potentially invoked (12.4 [class.dtor]).**

1949. “sequenced after” instead of “sequenced before”

Section: 1.9 [intro.execution] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-06-18

The term “sequenced after” is used in both the core and library clauses instead of the more-correct “sequenced before.”

Proposed resolution (May, 2015):

1. Change 1.9 [intro.execution] paragraph 13 as follows:

Sequenced before is an asymmetric, transitive, pair-wise relation between evaluations executed by a single thread (1.10 [intro.multithread]), which induces a partial order among those evaluations. Given any two evaluations *A* and *B*, if *A* is sequenced before *B* **(or, equivalently, *B* is sequenced after *A*)**, then the execution of *A* shall precede the execution of *B*. If *A* is not sequenced before *B*...

2. Change 1.10 [intro.multithread] paragraph 14 as follows:

An evaluation *A* *happens before* an evaluation *B* **(or, equivalently, *B* happens after *A*)** if...

3. Change 1.9 [intro.execution] paragraph 15 as follows:

...~~Every evaluation in the calling function (including other function calls) that is not otherwise specifically sequenced before or~~

after the execution of the body of the called function is indeterminately sequenced with respect to the execution of the called function. For each function invocation *F*, for every evaluation *A* that occurs within *F* and every evaluation *B* that does not occur within *F* but is evaluated on the same thread and as part of the same signal handler (if any), either *A* is sequenced before *B* or *B* is sequenced before *A*.⁹ [Note: if *A* and *B* would not otherwise be sequenced then they are indeterminately sequenced. —end note] Several contexts...

4. Change 3.6.2 [basic.start.init] paragraph 4 as follows:

It is implementation-defined whether the dynamic initialization of a non-local variable with static storage duration ~~is done~~ happens before the first statement of `main`. If the initialization is deferred to some point in time happen after the first statement of `main`, it shall occur happens before the first odr-use (3.2 [basic.def.odr]) of any function or variable...

5. Change 3.6.2 [basic.start.init] paragraph 5 as follows:

It is implementation-defined whether the dynamic initialization of a non-local variable with static or thread storage duration is done sequenced before the first statement of the initial function of the thread. If the initialization is deferred to some point in time sequenced after the first statement of the initial function of the thread, it shall occur is sequenced before the first odr-use (3.2 [basic.def.odr]) of any variable with thread storage duration defined in the same translation unit as the variable to be initialized.

6. Change 6.5.3 [stmt.for] paragraph 1 as follows:

...[Note: Thus the first statement specifies initialization for the loop; the condition (6.4 [stmt.select]) specifies a test, made sequenced before each iteration, such that the loop is exited when the condition becomes `false`; the expression often specifies incrementing that is done sequenced after each iteration. —end note]

7. Add the following as a new paragraph at the end of 15 [except]:

In this section, “before” and “after” refer to the “sequenced before” relation (1.9 [intro.execution]).

1975. Permissible declarations for *exception-specifications*

Section: 15.4 [except.spec] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-07-18

The declarations in which an *exception-specification* may appear is not completely clear from the current wording of 15.4 [except.spec] paragraph 2.

Suggested resolution:

Change 15.4 [except.spec] paragraph 2 as follows:

An *exception-specification* shall appear only on a function declarator for a function type, pointer to function type, reference to function type, or pointer to member function type that is the top-level type of a declaration or definition of a function, variable, or non-static data member, or on such a type appearing as a parameter or return type in such a function declarator. An *exception-specification* shall not appear in a typedef declaration or alias-declaration. [Example:...

See also issues 2010, 1995, 1946, and 1798.

Proposed resolution (May, 2015):

Change 15.4 [except.spec] paragraph 2 as follows:

An *exception-specification* shall appear only on within a lambda-declarator or within a function declarator for a function type, pointer to function type, reference to function type, or pointer to member function type that is the top-level type of a declaration or definition, or on such a type appearing as a parameter or return type in a function declarator. An *exception-specification* shall not appear in a typedef declaration or alias-declaration. of a function, variable, or non-static data member. It shall appear only on the top-level declarator, on the declarator of one of its parameter-declarations (if any), or on its return type. [Example:...

1981. Implicit contextual conversions and explicit

Section: 4 [conv] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-08-08

According to 4 [conv] paragraph 5,

Certain language constructs require conversion to a value having one of a specified set of types appropriate to the construct. An expression *e* of class type *ε* appearing in such a context is said to be *contextually implicitly converted* to a specified type *τ* and is well-formed if and only if *e* can be implicitly converted to a type *τ* that is determined as follows: *ε* is searched for conversion functions whose return type is *cv τ* or reference to *cv τ* such that *τ* is allowed by the context. There shall be exactly one such *τ*.

This description leaves open two questions: first, can `explicit` conversion functions be used for this conversion? Second, assuming that they cannot, is the restriction to “exactly one such τ ” enforced before or after exclusion of `explicit` conversion functions?

Notes from the November, 2014 meeting:

CWG felt that `explicit` conversion functions should be removed from consideration before determining the set of types for the conversion.

Proposed resolution (May, 2015):

Change 4 [conv] paragraph 5 as follows:

...An expression e of class type E appearing in such a context is said to be *contextually implicitly converted* to a specified type τ and is well-formed if and only if e can be implicitly converted to a type τ that is determined as follows: E is searched for **non-explicit** conversion functions whose return type is $cv\ \tau$ or reference to $cv\ \tau$ such that τ is allowed by the context. There shall be exactly one such τ .

1990. Ambiguity due to optional *decl-specifier-seq*

Section: 7 [dcl.dcl] **Status:** ready **Submitter:** Hubert Tong **Date:** 2014-08-27

In an example like

```
void f() {  
    f(); // #1  
}
```

The statement at #1 is ambiguous and can be parsed as either an expression or as a declaration. The problem is the fact that the *decl-specifier-seq* in a *simple-declaration* is optional.

Proposed resolution (May, 2015):

1. Change the grammar in 7 [dcl.dcl] paragraph 1 as follows:

```
declaration:  
    block-declaration  
    nodeclspec-function-declaration  
    function_definition  
...  
nodeclspec-function-declaration:  
    attribute-specifier-seqopt declarator ;  
  
alias-declaration:  
    using identifier attribute-specifier-seqopt = type-id ;  
  
simple-declaration:  
    decl-specifier-seqopt init-declarator-list ;  
    attribute-specifier-seq decl-specifier-seqopt init-declarator-list ;  
...
```

2. Change 7 [dcl.dcl] paragraph 2 as follows:

The A simple-declaration or nodeclspec-function-declaration of the form
attribute-specifier-seq_{opt} decl-specifier-seq_{opt} init-declarator-list_{opt} ;

is divided into three parts. Attributes are described in 7.6 [dcl.attr]. *decl-specifiers*, the principal components of a *decl-specifier-seq*, are described in 7.1 [dcl.spec]. *declarators*, the components of an *init-declarator-list*, are described in Clause 8 [dcl.dcl]. The *attribute-specifier-seq* **in a simple-declaration** appertains to each of the entities declared by the *declarators* of the *init-declarator-list*. [Note:...

3. Change 7 [dcl.dcl] paragraph 11 as follows:

Only in function declarations for A nodeclspec-function-declaration shall declare a constructors, destructors, and type or conversions **function** can the *decl-specifier-seq* be omitted.⁹³ **[Note: a nodeclspec-function-declaration can only be used in a template-declaration (Clause 14 [temp]), explicit-instantiation (14.7.2 [temp.explicit]), or explicit-specialization (14.7.3 [temp.expl.spec]). —end note]**

4. Change 8.3 [dcl.meaning] paragraph 1 as follows:

A list of declarators appears after an optional (Clause 7 [dcl.dcl]) decl-specifier-seq (7.1 [dcl.spec]). Each A declarator contains exactly one declarator-id; it names the identifier...

5. Change 12.1 [class.ctor] paragraph 1 as follows:

...~~In a constructor declaration, each~~ **Each** *decl-specifier* in the ~~optional~~ *decl-specifier-seq* **of a constructor declaration (if any)** shall be friend, inline, explicit, or constexpr. [Example:...

6. Change 12.3.2 [class.conv.fct] paragraph 1 as follows:

...Such functions are called conversion functions. ~~No return type can be specified.~~ **A *decl-specifier* in the *decl-specifier-seq* of a conversion function (if any) shall be neither a *type-specifier* nor *static*.** ~~If a conversion function is a member function, the~~ **The** type of the conversion function (8.3.5 [dcl.fct]) is...

7. Delete 12.3.2 [class.conv.fct] paragraph 6:

~~Conversion functions cannot be declared static.~~

8. Change 12.4 [class.dtor] paragraph 1 as follows:

...~~In a destructor declaration, each~~ **Each** *decl-specifier* of the ~~optional~~ *decl-specifier-seq* **of a destructor declaration (if any)** shall be friend, inline, or virtual.

This resolution also resolves issue 2016.

2000. *header-name* outside #include directive

Section: 2.4 [lex.pptoken] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-09-09

The “max munch” rule could be read to require the characters <int> in vector<int> to be parsed as a *header-name* rather than as three distinct tokens. 2.8 [lex.header] paragraph 1 says,

Header name preprocessing tokens shall only appear within a #include preprocessing directive (16.2 [cpp.include]).

However, that is not sufficiently clear that *header-names* are only to be recognized in that context.

Proposed resolution (May, 2015):

1. Change 2.4 [lex.pptoken] bullet 3.3 as follows:

- Otherwise, the next preprocessing token is the longest sequence of characters that could constitute a preprocessing token, even if that would cause further lexical analysis to fail, **except that a *header-name* (2.8 [lex.header]) is only formed within a #include directive (16.2 [cpp.include]).**

2. Change 2.8 [lex.header] paragraph 1 as follows:

[Note: Header name preprocessing tokens ~~shall~~ only appear within a #include preprocessing directive (16.2 [cpp.include] **see 2.4 [lex.pptoken]).** **—end note]** The sequences in both forms...

2004. Unions with mutable members in constant expressions

Section: 5.20 [expr.const] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-09-16

In an example like

```
union U { int a; mutable int b; };
constexpr U u1 = {1};
int k = (u1.b = 2);
constexpr U u2 = u1; // ok!!
```

The initialization of u2 is not disqualified by the current wording of the Standard because the copy is done via the object representation, not formally involving an lvalue-to-rvalue conversion. A restriction should be added to 5.20 [expr.const] forbidding the evaluation of a defaulted copy/move construction/assignment on a class type that has any variant mutable subobjects.

Proposed resolution (May, 2015):

1. Add the following bullet after bullet 3.1 of 7.1.5 [dcl.constexpr]:

The definition of a constexpr function shall satisfy the following constraints:

- it shall not be virtual (10.3 [class.virtual]);
- **for a defaulted copy/move assignment, the class of which it is a member shall not have a mutable subobject that is a variant member;**

- ...

2. Add the following bullet after bullet 4.1 of 7.1.5 [dcl.constexpr]

The definition of a `constexpr` constructor shall satisfy the following constraints:

- the class shall not have any virtual base classes;
- **for a defaulted copy/move constructor, the class shall not have a mutable subobject that is a variant member;**
- ...

2006. Cv-qualified void types

Section: 3.9.2 [basic.compound] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-09-16

According to 3.9.2 [basic.compound] paragraph 3,

The type of a pointer to `void` or a pointer to an object type is called an *object pointer type*. [Note: A pointer to `void` does not have a pointer-to-object type, however, because `void` is not an object type. —end note]

This wording excludes cv-qualified `void` types. There are other references in the Standard to “void type” that are apparently intended to include cv-qualified versions as well.

Proposed resolution (May, 2015):

1. Change 3.9 [basic.types] paragraph 5 as follows:

...Incompletely-defined object types and ~~the void types~~ **cv void** are incomplete types (3.9.1 [basic.fundamental])...

2. Change 3.9 [basic.types] paragraph 8 as follows:

An object type is a (possibly cv-qualified) type that is not a function type, not a reference type, and not ~~a void type~~ **cv void**.

3. Change 3.9.1 [basic.fundamental] paragraph 9 as follows:

~~The void type has an empty set of values. The~~ **A type cv void** ~~type~~ is an incomplete type that cannot be completed; **such a type has an empty set of values.** It is used as the return type for functions that do not return a value. Any expression can be explicitly converted to type `cv void` (5.4 [expr.cast]). An expression of type `cv void` shall be used only as an expression statement (6.2 [stmt.expr]), as an operand of a comma expression (5.19 [expr.comma]), as a second or third operand of `?:` (5.16 [expr.cond]), as the operand of `typeid`, `noexcept`, or `decltype`, as the expression in a return statement (6.6.3 [stmt.return]) for a function with the return type `cv void`, or as the operand of an explicit conversion to type `cv void`.

4. Change bullet 1.3 of 3.9.2 [basic.compound] as follows:

- pointers to **cv void** or objects or functions (including static members of classes) of a given type, 8.3.1 [dcl.ptr];

5. Change 3.9.2 [basic.compound] paragraph 3 as follows:

The type of a pointer to **cv void** or a pointer to an object type is called an *object pointer type*. [Note:...

2015. odr-use of deleted virtual functions

Section: 8.4.3 [dcl.fct.def.delete] **Status:** ready **Submitter:** David Majnemer **Date:** 2014-10-05

It is not clear that the odr-use of a virtual function described in 3.2 [basic.def.odr] paragraph 3 is exempt from the prohibition against referring to a deleted function (8.4.3 [dcl.fct.def.delete] paragraph 2).

Proposed resolution (May, 2015):

Change 8.4.3 [dcl.fct.def.delete] paragraph 2 as follows:

A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed. [Note: This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. If a function is overloaded, it is referenced only if the function is selected by overload resolution. **The implicit odr-use (3.2 [basic.def.odr]) of a virtual function does not, by itself, constitute a reference.** —end note]

2016. Confusing wording in description of conversion function

Section: 12.3.2 [class.conv.fct] **Status:** ready **Submitter:** Daniel Krügler **Date:** 2014-10-05

The statement in 12.3.1 [class.conv.ctor] paragraph 1,

No return type can be specified.

is confusing, since a conversion operator has a return type. It would be more precise to phrase the restriction in terms of the permissible *decl-specifiers* in the function's *decl-specifier-seq*. The next sentence is also problematic,

If a conversion function is a member function, the type of the conversion function (8.3.5 [dcl.fct]) is “function taking no parameter returning *conversion-type-id*”.

as it implies that a conversion function might not be a member function.

Proposed resolution (May, 2015):

This issue is resolved by the resolution of issue 1990.

2019. Member references omitted from description of storage duration

Section: 3.7.5 [basic.stc.inherit] **Status:** ready **Submitter:** David Krauss **Date:** 2014-10-08

According to 3.7.5 [basic.stc.inherit] paragraph 1,

The storage duration of member subobjects, base class subobjects and array elements is that of their complete object (1.8 [intro.object]).

This wording does not cover member references, which should also have the same storage duration as the object of which they are a member.

Proposed resolution (May, 2015):

Change 3.7.5 [basic.stc.inherit] paragraph 1 as follows:

The storage duration of member subobjects, base class subobjects and array elements **reference members** is that of their complete object (1.8 [intro.object]).

2024. Dependent types and unexpanded parameter packs

Section: 14.6.2.1 [temp.dep.type] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-10-17

Consider an example like:

```
template<typename ...Ts> struct X { X(int); };
template<typename T> using Y = int;
template<typename ...Ts> void f() {
    X<Y<Ts>...> x;
}
```

The presence of the ellipsis should make the reference to x a dependent type, but there is no rule making it so.

Proposed resolution (May, 2015):

Change 14.6.2.1 [temp.dep.type] paragraph 9 as follows:

A type is dependent if it is

- ...
 - a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent **or is a pack expansion**, or
-

2026. Zero-initialization and constexpr

Section: 3.6 [basic.start] **Status:** ready **Submitter:** Jason Merrill **Date:** 2014-10-20

According to 3.6.2 [basic.start.init] paragraph 2,

Variables with static storage duration (3.7.1 [basic.stc.static]) or thread storage duration (3.7.2 [basic.stc.thread]) shall be zero-initialized (8.5 [dcl.init]) before any other initialization takes place.

Does this apply to constant initialization as well? For example, should the following be well-formed, relying on the presumed zero-initialization preceding the constant initialization?

```
constexpr int i = i;
struct s {
    constexpr s() : v(v) { }
    int v;
};
constexpr s s1;
```

Notes from the November, 2014 meeting:

CWG agreed that constant initialization should be considered as happening instead of zero initialization in these cases, making the declarations ill-formed.

Proposed resolution (May, 2015):

1. Rename 3.6.2 [basic.start.init] and make the indicated changes, moving parts of its content to a new section immediately following, as indicated below:

3.6.2 Static initialization of non-local variables [basic.start.init.static]

There are two broad classes of named non-local variables: those with static storage duration (3.7.1 [basic.stc.static]) and those with thread storage duration (3.7.2 [basic.stc.thread]). Non-local variables **Variables** with static storage duration are initialized as a consequence of program initiation. Non-local variables **Variables** with thread storage duration are initialized as a consequence of thread execution. Within each of these phases of initiation, initialization occurs as follows.

Variables with static storage duration (3.7.1 [basic.stc.static]) or thread storage duration (3.7.2 [basic.stc.thread]) shall be zero-initialized (8.5 [dcl.init]) before any other initialization takes place. A *constant initializer* for an object *o* is an expression that is a constant expression, except that it may also invoke `constexpr` constructors for *o* and its subobjects even if those objects are of non-literal class types [Note: such a class may have a non-trivial destructor —end note]. *Constant initialization* is performed:

- if each full-expression (including implicit conversions) that appears in the initializer of a reference with static or thread storage duration is a constant expression (5.20 [expr.const]) and the reference is bound to a glvalue designating an object with static storage duration, to a temporary object (see 12.2 [class.temporary]) or subobject thereof, or to a function;
- if an object with static or thread storage duration is initialized by a constructor call, and if the initialization full-expression is a constant initializer for the object;
- if an object with static or thread storage duration is not initialized by a constructor call and if either the object is value-initialized or every full-expression that appears in its initializer is a constant expression.

If constant initialization is not performed, a variable with static storage duration (3.7.1 [basic.stc.static]) or thread storage duration (3.7.2 [basic.stc.thread]) is zero-initialized (8.5 [dcl.init]). Together, zero-initialization and constant initialization are called *static initialization*; all other initialization is *dynamic initialization*. Static initialization shall be performed before any dynamic initialization takes place. Dynamic initialization of a non-local variable with static storage duration is *unordered* if the variable is an implicitly or explicitly instantiated specialization, and otherwise is *ordered* [Note: an explicitly specialized static data member or variable template specialization has ordered initialization. —end note]. Variables with ordered initialization defined within a single translation unit shall be initialized in the order of their definitions in the translation unit. If a program starts a thread (30.3 [thread.threads]), the subsequent initialization of a variable is unsequenced with respect to the initialization of a variable defined in a different translation unit. Otherwise, the initialization of a variable is indeterminately sequenced with respect to the initialization of a variable defined in a different translation unit. If a program starts a thread, the subsequent unordered initialization of a variable is unsequenced with respect to every other dynamic initialization. Otherwise, the unordered initialization of a variable is indeterminately sequenced with respect to every other dynamic initialization. [Note: This definition permits initialization of a sequence of ordered variables concurrently with another sequence. —end note] [Note: The **dynamic** initialization of **non-local variables is described in 3.6.3 [basic.start.dynamic]; that of local static variables is described in 6.7 [stmt.dcl]. —end note]**

An implementation is permitted to perform the initialization of a **non-local** variable with static **or thread** storage duration as a static initialization even if such initialization is not required to be done statically, provided that

- the dynamic version of the initialization does not change the value of any other object of **namespace scope static or thread storage duration** prior to its initialization, and
- the static version of the initialization produces the same value in the initialized variable as would be produced by the dynamic initialization if all variables not required to be initialized statically were initialized dynamically.

[Note: As a consequence, if the initialization of an object `obj1` refers to an object `obj2` of namespace scope potentially requiring dynamic initialization and defined later in the same translation unit, it is unspecified whether the value of `obj2` used will be the value of the fully initialized `obj2` (because `obj2` was statically initialized) or will be the value of `obj2` merely zero-initialized. For example,

```
inline double fd() { return 1.0; }
```

```
extern double d1;
double d2 = d1; // unspecified:
                // may be statically initialized to 0.0 or
                // dynamically initialized to 0.0 if d1 is
                // dynamically initialized, or 1.0 otherwise
double d1 = fd(); // may be initialized statically or dynamically to 1.0
```

—end note]

2. Insert a new section after 3.6.2 [basic.start.init]:

3.6.3 Dynamic initialization of non-local variables [basic.start.dynamic]

3. Move part of 3.6.2 [basic.start.init] paragraph 2 as paragraph 1 of the new section:

Dynamic initialization of a non-local variable with static storage duration is unordered if the variable is an implicitly or explicitly instantiated specialization, and otherwise is ordered [Note: an explicitly specialized static data member or variable template specialization has ordered initialization. —end note]. Variables with ordered initialization defined within a single translation unit shall be initialized in the order of their definitions in the translation unit. If a program starts a thread (30.3 [thread.threads]), the subsequent initialization of a variable is unsequenced with respect to the initialization of a variable defined in a different translation unit. Otherwise, the initialization of a variable is indeterminately sequenced with respect to the initialization of a variable defined in a different translation unit. If a program starts a thread, the subsequent unordered initialization of a variable is unsequenced with respect to every other dynamic initialization. Otherwise, the unordered initialization of a variable is indeterminately sequenced with respect to every other dynamic initialization. [Note: This definition permits initialization of a sequence of ordered variables concurrently with another sequence. —end note]

4. Move paragraphs 4-6 of 3.6.2 [basic.start.init] as paragraphs 2-4 of the new section:

It is implementation-defined whether the dynamic initialization of a non-local variable with static storage duration is done before the first statement of `main`. If the initialization is deferred to some point in time after the first statement of `main`, it shall occur before the first odr-use (3.2 [basic.def.odr]) of any function or variable defined in the same translation unit as the variable to be initialized. [Footnote: A non-local variable with static storage duration having initialization with side-effects must be initialized even if it is not odr-used (3.2 [basic.def.odr], 3.7.1 [basic.stc.static]). —end footnote]
[Example:

```
// - File 1 -
```

```
#include "a.h"
#include "b.h"
B b;
A::A(){
    b.Use();
}
```

```
// - File 2 -
```

```
#include "a.h"
A a;
```

```
// - File 3 -
```

```
#include "a.h"
#include "b.h"
extern A a;
extern B b;
int main() {
    a.Use();
    b.Use();
}
```

It is implementation-defined whether either `a` or `b` is initialized before `main` is entered or whether the initializations are delayed until `a` is first odr-used in `main`. In particular, if `a` is initialized before `main` is entered, it is not guaranteed that `b` will be initialized before it is odr-used by the initialization of `a`, that is, before `A::A` is called. If, however, `a` is initialized at some point after the first statement of `main`, `b` will be initialized prior to its use in `A::A`. —end example]

It is implementation-defined whether the dynamic initialization of a non-local variable with static or thread storage duration is done before the first statement of the initial function of the thread. If the initialization is deferred to some point in time after the first statement of the initial function of the thread, it shall occur before the first odr-use (3.2 [basic.def.odr]) of any variable with thread storage duration defined in the same translation unit as the variable to be initialized.

If the initialization of a non-local variable with static or thread storage duration exits via an exception, `std::terminate` is called (15.5.1 [except.terminate]).

5. Change 6.7 [stmt.dcl] paragraph 4 as follows:

The zero-initialization (8.5 [dcl.init]) of all block-scope variables with static storage duration (3.7.1 [basic.stc.static]) or thread storage duration (3.7.2 [basic.stc.thread]) is performed before any other initialization takes place. Constant initialization (3.6.2 [basic.start.init]) of a block-scope entity with static storage duration, if applicable, is performed before its block is first entered.

An implementation is permitted to perform early initialization of other block-scope variables with static or thread storage duration under the same conditions that an implementation is permitted to statically initialize a variable with static or thread storage duration in namespace scope (3.6.2 [basic.start.init]). Otherwise such a variable is initialized **Dynamic initialization of a block-scope variable with static storage duration (3.7.1 [basic.stc.static]) or thread storage duration (3.7.2 [basic.stc.thread]) is performed** the first time control passes...

Editing note: all existing cross-references to 3.6.2 [basic.start.init] must be examined to determine which of the two current sections should be targeted.

2027. Unclear requirements for multiple `alignas` specifiers

Section: 7.6.2 [dcl.align] **Status:** ready **Submitter:** Steve Clamage **Date:** 2014-10-20

The description of *alignment-specifiers* is unclear. For example, 7.6.2 [dcl.align] bullet 2.2 says,

if the constant expression evaluates to a fundamental alignment, the alignment requirement of the declared entity shall be the specified fundamental alignment

However, paragraph 4 says,

When multiple *alignment-specifiers* are specified for an entity, the alignment requirement shall be set to the strictest specified alignment.

meaning that a less-strict alignment will be ignored, rather than being the alignment of the entity, and presumably meaning that no diagnostic is required for an insufficiently-strict alignment if a more stringent requirement is also supplied.

Proposed resolution (May, 2015):

1. Change 7.6.2 [dcl.align] paragraph 2 as follows:

When the *alignment-specifier* is of the form `alignas( constant-expression )`:

- the *constant-expression* shall be an integral constant expression;
- if the constant expression evaluates to a fundamental alignment, the alignment requirement of the declared entity shall be the specified fundamental alignment
- if the constant expression evaluates to an extended alignment and the implementation supports that alignment in the context of the declaration, the alignment of the declared entity shall be that alignment
- if the constant expression **does not evaluate to an alignment value (3.11 [basic.align]), or** evaluates to an extended alignment and the implementation does not support that alignment in the context of the declaration, the program is ill-formed.
- if the constant expression evaluates to zero, the alignment specifier shall have no effect
- otherwise, the program is ill-formed.

2. Change 7.6.2 [dcl.align] paragraph 4 as follows:

When multiple *alignment-specifiers* are specified for an entity, the **The** alignment requirement **shall be set to** of an entity is the strictest specified **non-zero** alignment **specified by its alignment-specifiers, if any; otherwise, the alignment-specifiers have no effect.**

2031. Missing incompatibility for `&&`

Section: C.2.3 [diff.cpp03.expr] **Status:** ready **Submitter:** Melissa Mears **Date:** 2014-10-31

The introduction of rvalue references in C++11 changed the interpretation of some previously well-formed examples such as the following:

```
struct Struct { template <typename T> operator T(); };
bool example_1 = new int && false;           // #1
bool example_2 = &Struct::operator int && false; // #2
```

Previously the `&&` was interpreted as an operator, while it is now part of a *type-name*. However, this change is not mentioned in Annex C [diff].

Proposed resolution (May, 2015):

Add the following as a new subsection in C.2.3 [diff.cpp03.expr]:

5.15 [expr.log.or]

Change: && is valid in a *type-name*

Rationale: Required for new features

Effect on original feature: Valid C++ 2003 code may fail to compile or produce different results in this International Standard, as the following example illustrates:

```
bool b1 = new int && false;           // previously false, now ill-formed
struct S { operator int(); };
bool b2 = &S::operator int && false; // previously false, now ill-formed
```

2052. Template argument deduction vs overloaded operators

Section: 13.5 [over.oper] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-12-03

In an example like

```
struct A { operator int(); };
template<typename T> T operator<<(T, int);
void f(A a) { 1 << a; }
```

Template argument deduction succeeds for the operator template, producing the signature `operator<<(int, int)`. The resulting declaration is synthesized and added to the overload set, per 14.8.3 [temp.over] paragraph 1. However, this violates the requirement of 13.5 [over.oper] paragraph 6,

An operator function shall either be a non-static member function or be a non-member function that has at least one parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration.

This is not a SFINAE context, so the program is ill-formed, rather than selecting the built-in operator.

Proposed resolution (May, 2015):

Change 14.8.3 [temp.over] paragraph 1 as follows,

...If, for a given function template, argument deduction fails **or the synthesized function template specialization would be ill-formed**, no such function is added to the set of candidate functions for that template.

2075. Passing short initializer lists to array reference parameters

Section: 13.3.3.1.5 [over.ics.list] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-01-27

According to 13.3.3.1.5 [over.ics.list] paragraph 5 says,

Otherwise, if the parameter type is “array of $N \times$ ”, if the initializer list has exactly N elements or if it has fewer than N elements and x is default-constructible, and if all the elements of the initializer list can be implicitly converted to x , the implicit conversion sequence is the worst conversion necessary to convert an element of the list to x .

There are terminological problems with this formulation. “Default constructible” is not otherwise used in the Standard and should instead refer to an implicit conversion sequence from an empty *braced-init-list*. Similarly, “can be implicitly converted” should instead refer to the existence of an implicit conversion sequence.

Proposed resolution, May, 2015:

Replace the indicated wording with:

Otherwise, if the parameter type is “array of $N \times$,” if there exists an implicit conversion sequence for each element of the array from the corresponding element of the initializer list (or from `{}` if there is no such element), the implicit conversion sequence is the worst such implicit conversion sequence.

2101. Incorrect description of type- and value-dependence

Section: 14.6.2 [temp.dep] **Status:** ready **Submitter:** Maxim Kartashev **Date:** 2015-03-16

According to 14.6.2 [temp.dep] paragraph 1,

Expressions may be *type-dependent* (on the type of a template parameter) or *value-dependent* (on the value of a non-type template parameter).

Proposed resolution (May, 2015):

Change 14.6.2 [temp.dep] paragraph 1 to read,

An expression may be *type-dependent* (that is, its type may depend on a template parameter) or *value-dependent* (that is, its value when evaluated as a constant expression (5.20 [expr.const]) may depend on a template parameter) as described in this subclause."

2120. Array as first non-static data member in standard-layout class

Section: 9 [class] **Status:** ready **Submitter:** Hubert Tong **Date:** 2015-05-06

The specification for determining whether a derived class is a standard-layout class, considering the types of its bases and those associated with its initial member, overlooked the case of a class whose first member is an array.

Proposed resolution (May, 2015):

Change 9 [class] paragraph 7 as follows:

... $M(x)$ is defined as follows:

- If x is a non-union class type, the set $M(x)$ is empty if x has no (possibly inherited (Clause 10 [class.derived])) non-static data members; otherwise, it consists of the type of the first non-static data member of x (where said member may be an anonymous union), x_θ , and the elements of $M(x_\theta)$.
- If x is a union type, the set $M(x)$ is the union of all $M(u_i)$ and the set containing all u_i , where each u_i is the type of the i th non-static data member of x .
- **If x is an array type with element type x_e , the set $M(x)$ consists of x_e and the elements of $M(x_e)$.**
- If x is a non-class, **non-array** type, the set $M(x)$ is empty.

[Note: $M(x)$ is the set of the types of all non-base-class subobjects that are guaranteed in a standard-layout class to be at a zero offset in x . —end note]